

Security Audit

YO (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The YO Protocol partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

YO Protocol's YoVault is an operator-managed, ERC-4626-compatible vault that extends standard vault functionality with an asynchronous redemption mechanism. Users deposit an underlying asset into the vault and receive vault shares in return. When a user requests redemption, the request is settled immediately if the vault has sufficient unreserved liquidity. Otherwise, the user's shares are transferred into escrow and added to an aggregated pending redemption entry associated with the selected receiver. An authorised operator is responsible for subsequently fulfilling or cancelling that pending entry.

Project Name: YO Protocol core-v2

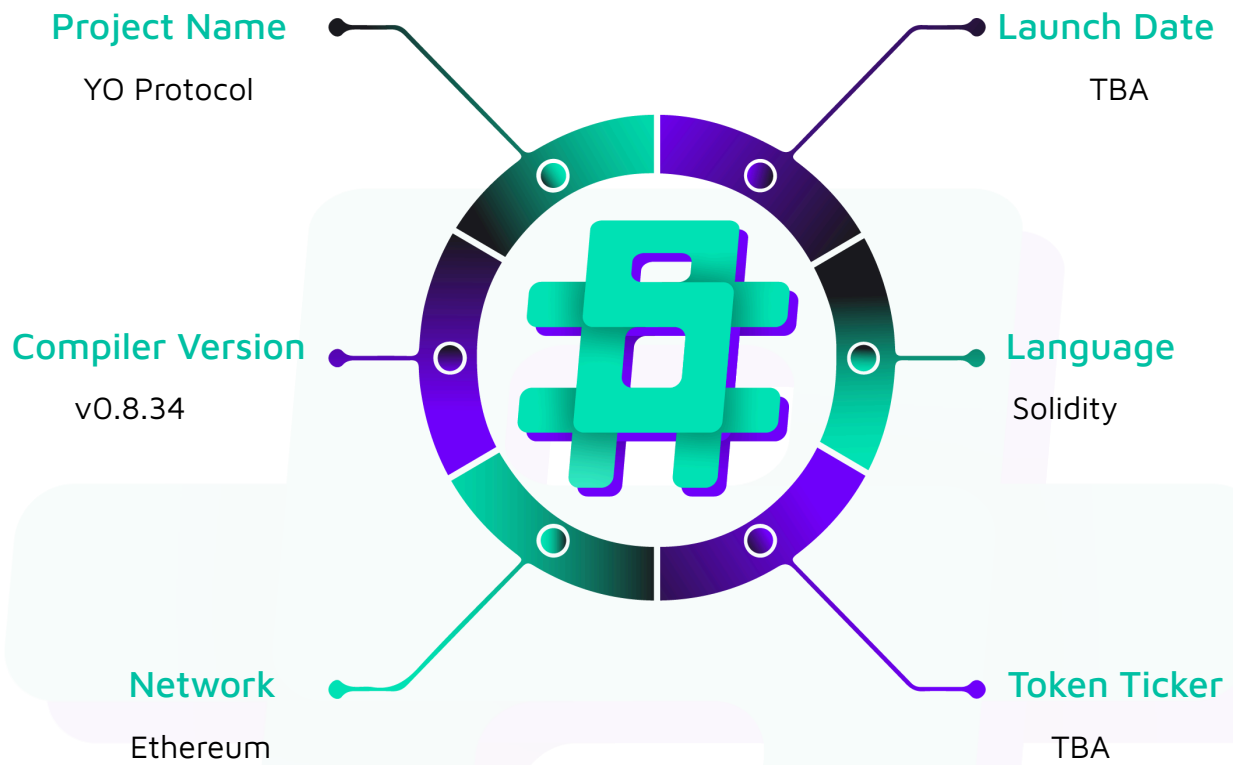
Project Type: Defi, ERC4626 vault, asynchronous redemption queue

Compiler Version: 0.8.34

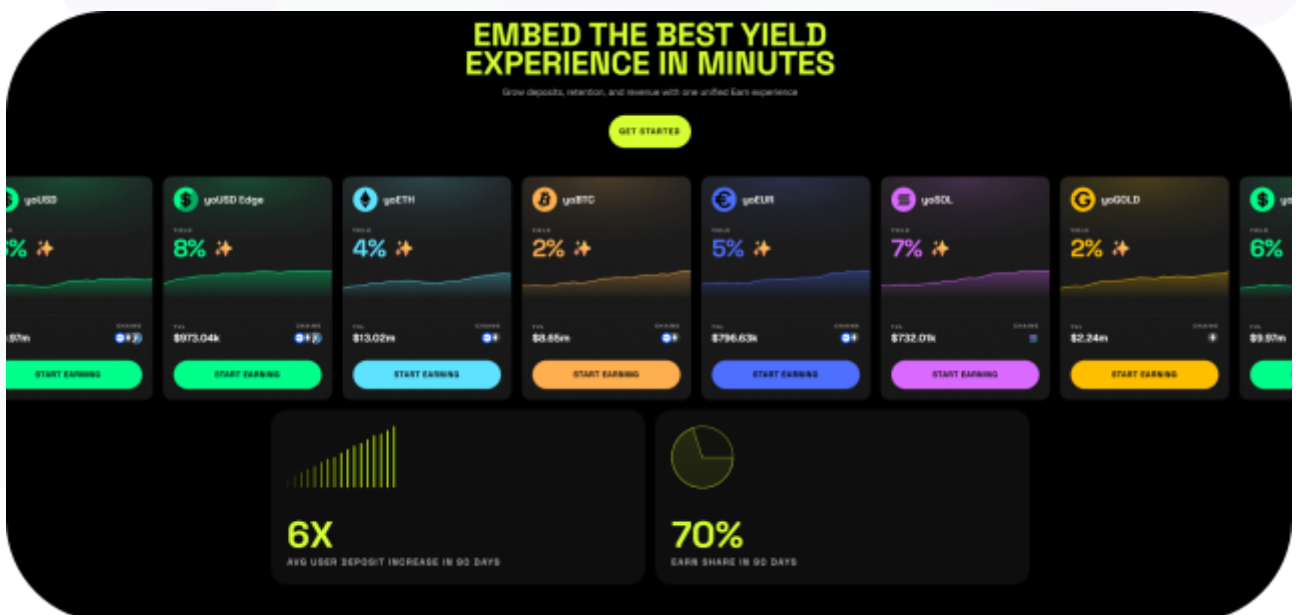
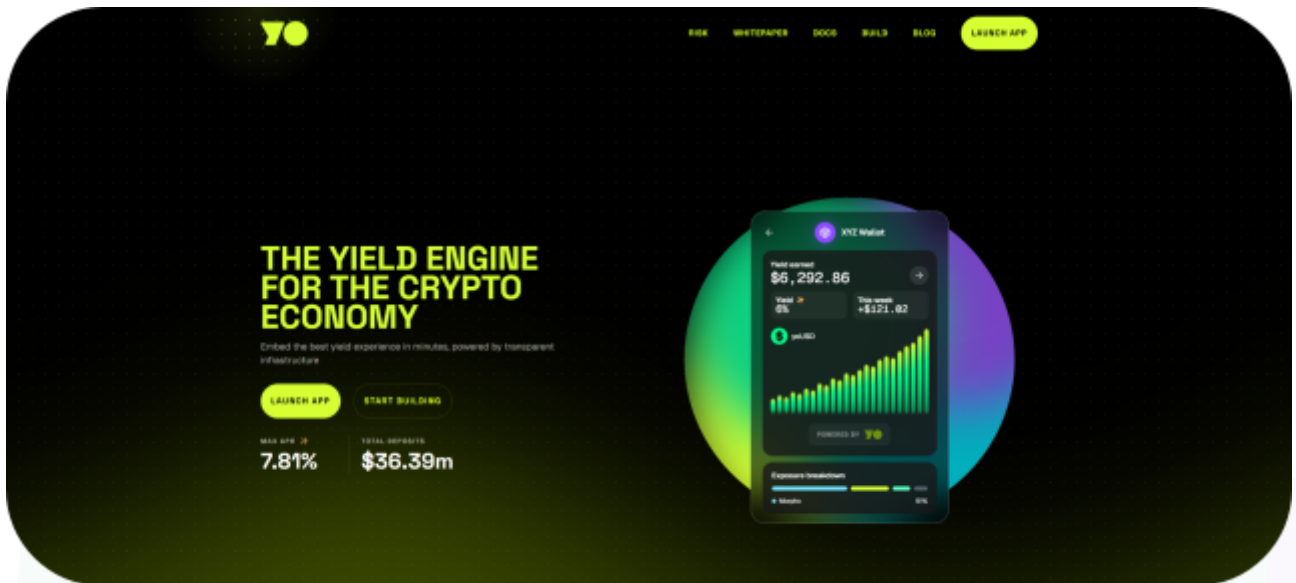
Website: <https://www.yo.xyz/>

Logo:



Visualised Context:

Project Visuals



Audit Scope

We at Hashlock audited the solidity code within the YO Protocol core-v2 , the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Template Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	September, 2026
Contract 1	YoVault.sol
Contract 2	Errors.sol
Audited GitHub Commit Hash	76e576bceb6875f6d59fbd7c8311f29f698e809b

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

2 Low severity vulnerabilities

1 QA issue

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
YoVault.sol - Allows users to request redemptions, settling immediately when sufficient unreserved liquidity is available and otherwise escrowing shares in a receiver-based pending entry. - Allows authorized callers to partially fulfil a pending redemption using a caller-selected share amount.	Contract achieves this functionality.
Errors.sol - Defines errors for invalid share slices, zero-gross fulfilment slices, self-receiver requests, and current prices above the reserved request value.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the YO Protocol, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the YO protocol smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] `src/YoVault.sol#fulfillRedeem/cancelRedeem` - Authorized settlement transactions are not bound to the pending entry they were created for

Description

`fulfillRedeem()` and `cancelRedeem()` operate on the receiver's pending state at execution time without verifying that it is the same entry the operator, Safe (or co-signers) reviewed when the transaction was prepared.

Vulnerability Details

All pending redemptions use the same `REQUEST_ID` and are aggregated only by the receiver. Each queued request increments `_pendingRedeem[receiver].shares` and `_pendingRedeem[receiver].assets`, including requests submitted by other share owners for that receiver. The new settlement functions contain no request nonce, deadline, expected share balance, or expected reserved-asset balance. `cancelRedeem(receiver)` deletes the whole entry that exists when the transaction executes, while `fulfillRedeem(receiver, shares, atCurrentPrice)` only verifies that the current entry contains at least the supplied number of shares.

Consequently, a delayed or pre-signed authorized transaction can cross pending-entry generations. For example, after a cancellation is approved, another share owner can append a request for the same receiver before execution, causing the cancellation to return every share in the enlarged entry. If the original entry is cleared and a new entry is later created for the same receiver, an old cancellation or fulfillment transaction can act on the replacement entry. An old current-price fulfillment can also apply a haircut to shares that were not part of the state reviewed by the signer.

Impact

Safe or co-signer transactions can unexpectedly cancel or settle newly appended or replacement redemption requests. This can return shares instead of assets or delay a withdrawal.

Recommendation

Bind every settlement instruction to the state that was reviewed. Include `expectedPendingShares` and `expectedPendingAssets` and require exact equality, or add a per-receiver request-generation nonce that changes whenever an empty entry becomes nonempty and whenever an entry is cleared.

Status

Acknowledged

[L-02] `src/YoVault.sol#cancelRedeem/fulfillRedeem` - Legacy self-receiver requests are not safely recoverable after the upgrade

Description

Currently, the upgrade blocks new redemption requests whose receiver is the vault itself, but it doesn't migrate or safely handle any self-receiver entry that was created under the previous implementation.

Vulnerability Details

The previous implementation didn't reject `receiver == address(this)`. Because the upgrade preserves the existing storage without an initializer or migration, `_pendingRedeem[address(vault)]` can already contain an in-flight request when the new implementation is activated. The problem is that, if the operator calls `cancelRedeem(address(vault))`, the function deletes the pending entry and releases its full asset reservation before transferring the escrowed shares from the vault to itself. The self-transfer doesn't return the shares to the original owner, so the shares remain in the vault after they are no longer represented by any pending entry.

If the operator instead calls `fulfillRedeem(address(vault), shares, ...)`, the escrowed shares are burned while the underlying asset transfer also names the vault as receiver. The assets therefore remain in the vault and the original request owner receives no payout. Because the pending state is keyed only by the receiver, the original owner cannot be recovered from storage after the upgrade.

Impact

An affected in-flight V2 request can be cleared while its shares become orphaned or burned and its economic value remains in the vault. Recovery then requires historical event reconstruction and a manual governance action.

Recommendation

Before upgrading each proxy, query `pendingRedeemRequest(address(proxy))`. If the entry is nonzero, resolve it under the previous implementation or use a dedicated migration/recovery path that reconstructs the original owner from historical events. The

upgrade script should revert while such an entry exists, and the test suite should include an old-implementation-to-new-implementation test covering both cancellation and fulfillment of legacy self-receiver state.

Status

Acknowledged



QA

[Q-01] `src/YoVault.sol#deposit/mint/_update` - Direct share transfers break the exact escrow accounting invariant

Description

The hand-off and the new invariant suite treat the vault's share balance as exactly equal to the sum of pending redemption shares, but public ERC-20 and ERC-4626 paths allow unrelated shares to be sent or minted directly to the vault.

Vulnerability Details

Queued redemption shares are escrowed through `_transfer(owner, address(this), shares)`, but the contract does not maintain a separate `totalPendingShares` ledger. The invariant suite therefore uses `balanceOf(address(vault))` as the escrow amount and asserts that it always equals the sum of `_pendingRedeem[receiver].shares`. However, `deposit()` and `mint()` don't reject `address(vault)` as the receiver, and `_update()` only checks whether the vault is paused. An existing shareholder can also call the ordinary ERC-20 `transfer()` function with the vault as recipient. Each of these operations increases `balanceOf(address(vault))` without creating or increasing any pending redemption entry. The equality consequently doesn't hold across the contract's full public state machine. Once unrelated shares reach the vault, their owner and purpose cannot be reconstructed from the pending-redemption ledger, and balance-based escrow reconciliation can no longer distinguish pending shares from direct transfers or direct mints.

Impact

Monitoring that relies on the stated equality can report false accounting corruption or incorrectly treat unrelated vault-held shares as redemption escrow. The issue doesn't directly remove assets from the vault, but it invalidates a security invariant presented as holding at all times.

Recommendation

Track pending shares explicitly in a `totalPendingShares` ledger that is updated in `requestRedeem()`, `fulfillRedeem()`, and `cancelRedeem()`, and reconcile the per-receiver sum against that ledger. Treat `balanceOf(address(vault)) >= totalPendingShares` as the balance-level solvency check.

Status

Acknowledged

Centralisation

The YO Protocol values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the YO Protocol seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.